Київський національний університет

імені Тараса Шевченка

Звіт до лабораторної роботи 2:

”Пошук MIS у дереві”,

дисципліни: актуальні проблеми “Розподілені

системи обробки інформації”,

студента: Євчика Олексія,

групи Інформатика, Магістри

Тема

Пошук найбільшої незалежної множини у дереві.

Мета роботи

Дослідити задачу пошуку найбільшої незалежної множини у дереві та реалізувати алгоритм її знаходження. Визначити розмір множини та перелік вершин, що до неї входять.

Завдання роботи

1. Ознайомитися з поняттям незалежної множини графа.
2. Розглянути особливості задачі для дерева.
3. Представити дерево у вигляді структури даних (список суміжності).
4. Реалізувати алгоритм пошуку найбільшої незалежної множини.
5. Обчислити розмір MIS.
6. Визначити вершини, що входять до MIS.
7. Протестувати алгоритм на різних деревах.
8. Вивести:
    a. розмір MIS
    b. список вершин MIS

Теоретичні відомості

Незалежна множина (Independent Set, IS) у графі $G = (V, E)$ - це підмножина вершин $S \subseteq V$, така що для будь-яких двох вершин $u, v \in S$ ребро $(u, v) \notin E$. Іншими словами, жодні дві вершини з S не є суміжними.

Найбільша незалежна множина (Maximum Independent Set, MIS) - незалежна множина максимально можливого розміру. Параметр $\alpha(G) = |MIS|$ називають числом незалежності графа.

Для довільного графа задача знаходження MIS є **NP-складною**. Однак для спеціальних класів графів вона розв'язується ефективно. Зокрема, для дерев існує лінійний алгоритм на основі динамічного програмування.

Особливості задачі для дерева

Дерево T = (V, E) - зв'язний ациклічний граф. Завдяки відсутності циклів та ієрархічній структурі задача MIS на дереві допускає оптимальну підструктуру: значення для піддерева з коренем у вершині v залежить лише від безпосередніх нащадків v. Це дозволяє застосувати динамічне програмування.

Динамічне програмування на дереві

Фіксуємо довільну вершину як корінь. Для кожної вершини v вводимо два значення:

- **dp[v][1]** - розмір MIS у піддереві з коренем v, якщо v включено до множини;
- **dp[v][0]** - розмір MIS у піддереві з коренем v, якщо v не включено до множини.

Рекурентні співвідношення:

dp[v][1] = 1 + sum(dp[u][0]) для всіх дітей u вершини v

dp[v][0] = sum(max(dp[u][0], dp[u][1])) для всіх дітей u вершини v

Якщо v - листова вершина: dp[v][1] = 1, dp[v][0] = 0.

Відповідь: **max(dp[root][0], dp[root][1])**. Відновлення множини виконується жадібним проходом зверху вниз: якщо dp[v][1] ≥ dp[v][0], вершина v включається, а її діти - ні.

Складність алгоритму

Характеристика	Значення
Часова складність	O(n)
Просторова складність	O(n)
Клас задачі	P (поліноміальний час)
Застосовність	Тільки дерева (ациклічні графи)

Алгоритм (опис)

1. Представити дерево у вигляді списку суміжності.
2. Обрати довільну вершину як корінь.
3. Виконати DFS обхід дерева.
4. Для кожної вершини v обчислити:
   - include[v] = 1
   - exclude[v] = 0
5. Для кожної дитини child вершини v:
   - рекурсивно обчислити значення
   - include[v] += exclude[child]
   - exclude[v] += max(include[child], exclude[child])
6. Після обходу:
   - розмір MIS = max(include[root], exclude[root])
7. Для відновлення вершин MIS:
   - пройти дерево повторно
   - якщо include[v] > exclude[v] → додати вершину
   - тоді пропустити її дітей
   - інакше перейти до дітей
8. Вивести:
   - розмір MIS
   - список вершин, що входять до множини

Постановка задачі

Дано дерево з N вершинами. Необхідно знайти найбільшу незалежну множину вершин. Незалежною називається така множина вершин графа, в якій жодні дві вершини не з'єднані ребром.

Потрібно:

- визначити розмір найбільшої незалежної множини (MIS)
- визначити перелік вершин, що входять до цієї множини

Обмеження:

- граф є деревом
- дерево зв'язне та не містить циклів
- вершини нумеруються від 0 до N-1

Спосіб розв'язання

Для дерева задача розв'язується методом динамічного програмування з обходом у глибину (DFS).

Для кожної вершини v розглядаються два випадки:

- вершина входить до незалежної множини
- вершина не входить до незалежної множини

Вводяться два значення:

- include[v] - максимальний розмір MIS, якщо вершина v включена
- exclude[v] - максимальний розмір MIS, якщо вершина v не включена

Формули обчислення:

include[v] = 1 + Σ exclude[child]

exclude[v] = Σ max(include[child], exclude[child])

Обхід дерева виконується DFS від кореня.

Після завершення обходу:

MIS = max(include[root], exclude[root])

Для відновлення вершин множини виконується повторний обхід дерева.

Складність алгоритму:

- часова складність: O(n)
- просторова складність: O(n)

Деталі реалізації

Дерево представлено у вигляді списку суміжності.
Алгоритм складається з двох етапів:

1. DFS обхід для обчислення include та exclude
2. Відновлення вершин, що входять до MIS

Реалізація виконана мовою Python.

https://github.com/MoloZzz/distributed-systems-tests/tree/main/labs/lab2

Мова програмування та інструменти

Компонент	Опис
Мова	Python 3.10+
Структура даних	Список суміжності (collections.defaultdict)
Обхід графа	Ітеративний DFS (стек, без рекурсії)
Зовнішні бібліотеки	Відсутні (лише стандартна бібліотека)

Структура програми

Програма містить такі компоненти:

1. Клас Tree
    - ініціалізація вершини, кореня, імені
    - побудова списку суміжності
    - метод __repr__
2. Функція find_mis(tree: Tree) -> tuple
    - обхід дерева в пост-ордері
    - DP-обчислення MIS для кожної вершини
    - відновлення множини вершин MIS
3. Функція verify_is(tree: Tree, mis: list[int]) -> bool
    - перевірка, чи є mis справжньою незалежною множиною
4. Функція make_trees() -> list[Tree]
    - створення набору тестових дерев
5. Функція print_result(tree: Tree, size: int, mis: list[int]) -> None
    - вивід результатів і перевірка коректності

Ключові фрагменти коду

Ітеративний post-order обхід (уникнення переповнення стеку):

```
    stack = [tree.root]
    visited[tree.root] = True

    while stack:
        v = stack.pop()
        order.append(v)
        for u in tree.adj[v]:
            if not visited[u]:
                visited[u] = True
                parent[u] = v
                stack.append(u)
```

Обчислення DP (знизу вгору через reversed order):

```
for v in reversed(order):  # знизу вгору
    inc = 1                 # включаємо v
    exc = 0                 # виключаємо v
    for u in tree.adj[v]:
        if u == parent[v]:
            continue
        inc += dp[u][0]
        exc += max(dp[u][0], dp[u][1])
    dp[v][1] = inc
    dp[v][0] = exc
```

Особливості реалізації

- Ітеративний DFS замість рекурсивного дозволяє обробляти дерева великих розмірів без ризику переповнення стеку виконання.
- Список суміжності зберігається у вигляді defaultdict(list), що спрощує побудову графа.
- Фаза відновлення використовує масив force_out для передачі обмежень від батьків до дітей без зайвих рекурсивних викликів.
- Вбудована верифікація підтверджує коректність: для кожного ребра (u,v) перевіряється, що обидві вершини одночасно не входять до MIS.

Висновок

У ході роботи досліджено задачу пошуку найбільшої незалежної множини у дереві. Було використано метод динамічного програмування з обходом DFS. Реалізований алгоритм працює за лінійний час O(n) та використовує O(n) пам’яті. Програма визначає розмір найбільшої незалежної множини та перелік вершин, що до неї входять. Результати тестування підтвердили коректність роботи алгоритму.

Додаток (запуск коду)

```
$ python labs/lab2/mis-tree.py
Пошук найбільшої незалежної множини (MIS) у дереві
Алгоритм: DP на дереві (post-order), O(n)

============================================================
  Загальне дерево (12 вершин)
============================================================
  Вершин у дереві  : 12
  Розмір MIS       : 7
  Вершини MIS      : [1, 2, 7, 8, 9, 10, 11]
  Перевірка IS     : ✓ коректно

============================================================
  Зірка (9 вершин)
============================================================
  Вершин у дереві  : 9
  Розмір MIS       : 8
  Вершини MIS      : [1, 2, 3, 4, 5, 6, 7, 8]
  Перевірка IS     : ✓ коректно

============================================================
  Ланцюг (8 вершин)
============================================================
  Вершин у дереві  : 8
  Розмір MIS       : 4
  Вершини MIS      : [0, 2, 4, 6]
  Перевірка IS     : ✓ коректно

============================================================
  Повне бінарне дерево (15 вершин)
============================================================
  Вершин у дереві  : 15
  Розмір MIS       : 10
```

```
============================================================
  Повне бінарне дерево (15 вершин)
============================================================
  Вершин у дереві  : 15
  Розмір MIS       : 10
  Вершини MIS      : [1, 2, 7, 8, 9, 10, 11, 12, 13, 14]
  Перевірка IS     : ✓ коректно

============================================================
  Гусениця (10 вершин)
============================================================
  Вершин у дереві  : 10
  Розмір MIS       : 5
  Вершини MIS      : [0, 2, 4, 6, 8]
  Перевірка IS     : ✓ коректно

============================================================
  Одна вершина
============================================================
  Вершин у дереві  : 1
  Розмір MIS       : 1
  Вершини MIS      : [0]
  Перевірка IS     : ✓ коректно

============================================================
  Два вузли
============================================================
  Вершин у дереві  : 2
  Розмір MIS       : 1
  Вершини MIS      : [0]
  Перевірка IS     : ✓ коректно

============================================================
```